

Documentation for Running the Scaling Code

Code description

The code solves Boussinesq equations in N^3 grid points using MPI parallelisation. The time step size is fixed at $dt = 10^{-3}$. The code is executed for a total simulation time T , so that the total number of time steps is T/dt . A detailed step-by-step procedure for running the code is given below.

The total walltime taken by the code will be saved in a file named **Total Time_taken.txt**.

Steps to run the code

- Install the required Python packages: **jax**, **sympy**, and **tqdm**.
- Create the required directories for saving the simulation data according to the chosen saving frequency.
- To change the spatial resolution of the simulation, vary the variable **N** in the code. For example, setting **N = 512** gives a **512³** resolution.
- For a chosen resolution **N**, use a suitable number of MPI cores according to the decomposition requirements of the code.
- To vary the total number of time steps (and hence the total simulation time), vary the variable **T** in the code. The time step is fixed at $dt = 10^{-3}$, so the total number of simulation steps is **steps = T/dt**. For example, **T = 0.1** corresponds to 100 time steps.
- To check the speed/performance of the code with a high saving frequency, vary the variable **save_every**. A smaller value of **save_every** saves data more frequently and therefore includes more I/O overhead, which can be used to assess the effect of saving frequency on the measured runtime.